

Assignment -1

Implement all types of inheritance

Single inheritance:

Explanation:

1. **class hello** — the parent (superclass). It has a method `show()` that returns the string "Hello".
2. **class World extends hello** — the child (subclass). The `extends` keyword means `World` inherits all members of `hello`. It overrides `show()`, and inside it calls `super.show()` to get "Hello" from the parent, then appends " World" to it — producing "Hello World".
3. **super.show()** — this is the key inheritance concept: `super` refers to the immediate parent class, letting the child reuse the parent's logic instead of rewriting it.
4. **main method** — creates a `World` object, calls `show()` on it (which internally calls the parent's `show()` too), and prints the result: `Hello World`.

Code:

```

package inheritance;

//single level inheritance
class hello{      String
show(){          return
"Hello";
    } } class World extends hello{
String show(){    return
super.show()+ " World";
    }
} public class SingleInheritance {
public static void main(String[] args) {
    World w = new World();

    String s = w.show();
    System.out.println(s);
}
}

```

OUTPUT:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.SingleInheritance'
Hello World
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.SingleInheritance'
Hello World
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.SingleInheritance'
Hello World
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

Multilevel Inheritance:

Explanation:

This program demonstrates multilevel inheritance, where a class inherits from a class, which itself inherits from another class — forming a chain (College to Department to Student).

1. Top-level parent class (College): This is the base class in the chain. It has a method `getCollegeName()` that prints the college name. It does not inherit from anything.
2. Middle class (Department): This class extends College, making it both a child of College and a parent to the next class. It adds its own method `getDepartmentName()` while also inheriting `getCollegeName()` from College.
3. Bottom-level class (Student): This class extends Department. Since Department already inherited from College, Student ends up inheriting from both classes in the chain. It adds its own method `getStudentName()`, on top of the two inherited methods.
4. The inheritance chain: College to Department to Student. This is what makes it multilevel — there are multiple levels of inheritance stacked on top of each other, unlike singlelevel inheritance where there is only one parent-child relationship.
5. The main method: A single object of the Student class is created. Because of the inheritance chain, this one object can access all three methods: `getCollegeName()` from College, `getDepartmentName()` from Department, and `getStudentName()` from Student, even though only one class was directly instantiated.
6. Output: When the program runs, it prints all three lines in order:
College name is: ANITS College
Department name is: CSE(AIML)
Student name is: sushant singh rajput

Key takeaway: Multilevel inheritance allows a class to inherit properties and methods through a chain of classes, not just from one direct parent. A single object of the lowest class in the chain gains access to all the methods defined at every level above it, which shows how inheritance can be layered to build increasingly specialized classes.

Code:

```

package inheritance;

//parent class class
College{    void
getCollegeName(){
    System.out.println("College name is: ANITS College");
}
}

//class child1 inherits parent college class class
Department extends College{    void getDepartmentName(){
System.out.println("Department name is: CSE(AIIML)");
}
}

//class child2 inherits child1 class
class Student extends Department{
void getStudentName(){
    System.out.println("Student name is: sushant singh
rajput");
}
} public class
MultilevelInheritance {
    public static void main(String[] args) {
        //creating a single object for all classes to access
        details in respective classes        Student s = new
Student();
        s.getCollegeName();
        s.getDepartmentName();

        s.getStudentName();
    }
}

```

OUTPUT:

```

deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.MultilevelInheritance'
College name is: ANITS College
Department name is: CSE(AIPL)
Student name is: sushant singh rajput      c;; cd 'C:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-xx:+show
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.MultilevelInheritance'
College name is: ANITS College
Department name is: CSE(AIPL)
Student name is: sushant singh rajput
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

Hierarchical Inheritance Explanation:

This program demonstrates hierarchical inheritance, where multiple child classes inherit from a single parent class.

1. Parent class (College): This is the base class. It has a method `getCollegeName()` that prints the college name. All three child classes will inherit from this one class.
2. Child class 1 (Student1): This class extends College and adds its own method `getStudentName()`, which prints "Prabhash". It inherits `getCollegeName()` from College.
3. Child class 2 (Student2): This class also extends College independently and adds its own version of `getStudentName()`, which prints "Anushka". It too inherits `getCollegeName()` from College.
4. Child class 3 (Student3): This class also extends College and adds its own version of `getStudentName()`, which prints "Yash". It also inherits `getCollegeName()` from College.
5. The hierarchy: One parent (College) has three separate children (Student1, Student2, Student3), all branching out from it. This is what makes it hierarchical — unlike multilevel inheritance where classes form a chain, here multiple classes independently inherit from the same single parent.
6. The main method:
 - An object of Student1 is created and calls `getStudentName()`, printing "Prabhash".
 - An object of Student2 is created and calls both `getStudentName()` (printing "Anushka") and the inherited `getCollegeName()` (printing the college name).
 - An object of Student3 is created and calls `getStudentName()`, printing "Yash".
 - Finally, a parent class reference (College s4) is used to hold a Student1 object. This is called upcasting — a parent class reference can point to a child class object. Since s4 is declared as type College, it can only call `getCollegeName()` through this reference, not `getStudentName()`, because College doesn't know about that method.
7. Output: When the program runs, it prints:


```

Student name is: Prabhash
Student name is: Anushka
College name is: ANITS College
Student name is: Yash
College name is: ANITS College

```

Key takeaway: Hierarchical inheritance allows one parent class to be extended by multiple independent child classes, each adding its own specific behavior while sharing the common features of the parent. This is different from multilevel inheritance, where each class builds on the previous one in a chain rather than branching off a common base.

Code:

```
package inheritance;
//hierachial inheritance
class College {    void
getCollegeName() {
    System.out.println("College name is: ANITS College");
}
} class Student1 extends
College {    void
getStudentName() {
    System.out.println("Student name is: Prabhash");
}
} class Student2 extends
College {    void
getStudentName() {
    System.out.println("Student name is: Anushka");
}
} class Student3 extends
College {    void
getStudentName() {
    System.out.println("Student name is: Yash");
}
```

```

    }
}
// 3 classes inherits the same parent class
public class Hierachial {    public static
void main(String[] args) {
    //creating objects for different classes which
inherits from parent class

    Student1 s1 = new Student1();
s1.getStudentName();

    Student2 s2 = new Student2();
s2.getStudentName();        s2.getCollegeName();

    Student3 s3 = new Student3();
s3.getStudentName();

    //parent class object
College s4=new Student1();
s4.getCollegeName();
    }
}

```

OUTPUT:

```

College name is: ANITS College
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.Hierachial'
Student name is: Prabhask
Student name is: Anushka
College name is: ANITS College
Student name is: Yash
College name is: ANITS College
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.Hierachial'
Student name is: Prabhask
Student name is: Anushka
College name is: ANITS College
Student name is: Yash
College name is: ANITS College
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

Interface Implementation:

Explanation:

This program demonstrates the use of interfaces, where a class implements multiple interfaces to gain their abstract methods and then provides its own implementation for each of them.

1. Interface Cinema: This interface declares two methods, `filmindustry()` and `tollywood()`. Interface methods are by default abstract and public, meaning they have no body — any class that implements this interface must provide its own implementation for them.
2. Interface Director: This interface declares two more methods, `storywriter()` and `Actionmovies()`. Just like Cinema, these methods are abstract by default and must be implemented by any class that uses this interface.
3. Class Hero implements Director, Cinema: This is the key part of the program. A class can implement multiple interfaces at once (unlike extending classes, where Java allows only single inheritance). Here, Hero implements both Cinema and Director, meaning it must provide implementations for all four abstract methods declared across both interfaces.
4. Constructor (`Hero(String hero)`): When a Hero object is created, this constructor runs, sets the hero variable, and prints the hero's name.
5. Overridden methods: The class provides its own body for all four interface methods:
 - `filmindustry()` prints "Annapurna"
 - `tollywood()` prints "telugu films"
 - `storywriter()` prints "storywriter: sudhamurthy"
 - `Actionmovies()` prints "PARADISE"The `@Override` annotation confirms these methods are implementing the interface methods, not creating new ones.
6. Extra method (`heroine()`): This is a method defined only in the Hero class, not part of either interface. It simply prints "srinidhi shetty". This shows a class can have its own additional methods beyond what the interfaces require.
7. The main method:
 - A Hero object is created with the name "NANI", which triggers the constructor and prints "NANI".
 - `h.heroine()` is called, printing "srinidhi shetty".
 - `h.Actionmovies()` is called, printing "PARADISE".
 - `h.filmindustry()` is called, printing "Annapurna".
 - `h.tollywood()` is called, printing "telugu films".
 - `h.storywriter()` is called, printing "storywriter: sudhamurthy".
8. Output: When the program runs, it prints:
NANI
srinidhi shetty
PARADISE Annapurna
telugu films storywriter:
sudhamurthy
Key takeaway: An interface defines a contract of

abstract methods with no implementation. A class uses the implements keyword (instead of extends) to agree to that contract, and it must override every method declared in the interface(s). Unlike class inheritance, a single class can implement multiple interfaces at once, which is how Java achieves a form of multiple inheritance.

Code:

```
package inheritance;
//interface
interface Cinema {    void filmindustry();//all are
bydefault abstract methods    void tollywood();
} interface Director {
void storywriter();
void Actionmovies();
}

//class inherits interface (implements)
//interface must inherits class(override) to access
class Hero implements Director,Cinema{    String
hero;    public Hero(String hero){
this.hero=hero;
        System.out.println(hero);
    }
    @Override    public void
filmindustry(){
        System.out.println("Annapurna");
    }
    @Override    public
void tollywood(){
```

```

        System.out.println("telugu films");
    }
    @Override    public void
storywriter(){
        System.out.println("storywriter: sudhamurthy" );
    }
    @Override    public void
Actionmovies(){
        System.out.println("PARADISE");
    }    void
heroine(){
        System.out.println("srinidhi shetty");
    }
} public class Interface {    public
static void main(String[] args) {
Hero h=new Hero("NANI");
    h.heroine();
    h.Actionmovies();
    h.filmindustry();
    h.tollywood();
    h.storywriter();
    }
}

```

OUTPUT:

```

storywriter: sudhamurthy
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.Interface'
NANI
srinidhi shetty
PARADISE
Annapurna
telugu films
storywriter: sudhamurthy
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

hybrid inheritance

Hybrid inheritance

Explanation:

This program demonstrates hybrid inheritance, which is a combination of two or more types of inheritance — here, single inheritance and multiple inheritance (achieved through a class extending another class while also implementing an interface).

1. Interface I: This interface declares one abstract method, `getI()`. It has no body, so any class implementing it must override this method.
2. Class A: This is the base class. It has a method `getA()` that prints "A". It doesn't inherit from anything.
3. Class B extends A: This represents single inheritance — B inherits from only one class, A. It adds its own method `getB()` while also gaining access to `getA()` from A.
4. Class C extends A implements I: This is where the hybrid part comes in. C inherits from class A (extends) and also implements interface I (implements) at the same time. This combination — inheriting from a class and implementing an interface together — is what achieves multiple inheritance in Java, since Java doesn't allow a class to extend more than one class directly, but it does allow implementing multiple interfaces alongside a single class extension. C adds its own method `getC()` and also overrides `getI()` from the interface, printing "I".
5. Why it's called hybrid: The overall structure combines single inheritance (B extending A) with hierarchical/multiple inheritance (C extending A and implementing I). Mixing these different inheritance types together in one program is what makes it "hybrid".
6. The main method:
 - An object of B is created. It calls `getB()` (printing "B") and the inherited `getA()` (printing "A").
 - An object of C is created. It calls `getI()` (printing "I", from the interface) and `getC()` (printing "C", its own method). C also has access to `getA()` from class A, though it isn't called here.
7. Output: When the program runs, it prints:
B
A
I
C

Key takeaway: Hybrid inheritance combines multiple inheritance types in a single design — here, a class can extend another class (gaining its methods directly) while also implementing an interface (gaining a contract of abstract methods to fulfill). Since Java doesn't support multiple class inheritance directly, using interfaces alongside class extension is how Java achieves the effect of multiple inheritance within a hybrid structure.

Code:

```
package inheritance;  
//hybrid inheritance
```

```
interface I{
    void getI();//abstract method
}

class A{
    void getA(){
        System.out.println("A");
    }
}

//single inheritance
class B extends A{
    void getB(){
        System.out.println("B");
    }
}

//hierachial inheritance (multiple inheritance)
class C extends A implements I{
    void getC(){
        System.out.println("C");
    }
    @Override
    public void getI(){
        System.out.println("I");
    }
}

public class Hybrid {
    public static void main(String[] args) {
        //creting objects for class b and c

        B b=new B();
        b.getB();
        b.getA();
    }
}
```

```
C c=new C();  
c.getI();  
c.getC();  
  
}  
}
```

OUTPUT:

```
I  
C  
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'inheritance.Hybrid'  
B  
A  
I  
C  
PS C:\Users\Admin\OneDrive\Documents\basicjava> |
```